

Cross-Domain Misconfiguration (Systemic)

Assessment Identifier: **WS-DYN-05** | Assessment Type: **Dynamic** | Date: **September 08, 2026**

Severity: MEDIUM	CVSS: 5.3	CWE: CWE-264
OWASP: A01:2021 - Broken Access Control	WASC ID: 14	Confidence: Medium
Target / Component: Public Information Endpoint	Affected URL: https://www.worldmonitor.app/robots.txt	Parameter: Header Line: access-control-allow-origin: *

1. Description

Web browser data loading may be possible due to a Cross-Origin Resource Sharing (CORS) misconfiguration on the web server. The web server returns the Access-Control-Allow-Origin: * HTTP response header, which permits any arbitrary third-party domain to make cross-domain read requests and read responses via browser APIs (such as fetch or XMLHttpRequest).

2. Steps to Reproduce

1. Send an HTTP GET request to https://www.worldmonitor.app/robots.txt from a client or custom origin (e.g., using curl or browser dev tools with an Origin: https://evil.com header).
2. Inspect the returned HTTP response headers.
3. Observe that the response includes access-control-allow-origin: *.

3. Controlled Proof of Concept

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 10:28:22 GMT
Content-Type: text/plain; charset=utf-8
Connection: keep-alive
access-control-allow-origin: *
Age: 7557
Cache-Control: public, max-age=0, must-revalidate
content-disposition: inline; filename="robots.www.txt"

Browser Verification:
fetch('https://www.worldmonitor.app/robots.txt').then(res => res.text()).then(console.log);
```

4. Business & Security Impact

The CORS misconfiguration permits cross-domain read requests from arbitrary origins. On public resources like robots.txt, the immediate risk is low. However, if wildcard or overly permissive CORS policies are broadly applied across authenticated endpoints or sensitive internal APIs on the domain, attackers can craft malicious sites that extract private user data or perform authenticated cross-site data harvesting on behalf of logged-in victims.

5. Remediation Recommendations

Primary Fix: Restrict Access-Control-Allow-Origin to an explicit allowlist of trusted domains rather than using a wildcard (*).

1. Ensure sensitive or user-specific data is not exposed without authentication and explicit access controls.
2. Restrict the Access-Control-Allow-Origin header to an explicit whitelist of trusted domain names rather than using a wildcard (*).
3. If public resources (like static files or public assets) require cross-origin access, maintain CORS headers strictly on those specific routes and remove wildcard headers from sensitive API routes so the browser enforces the Same-Origin Policy (SOP).

6. Evidence & Validation

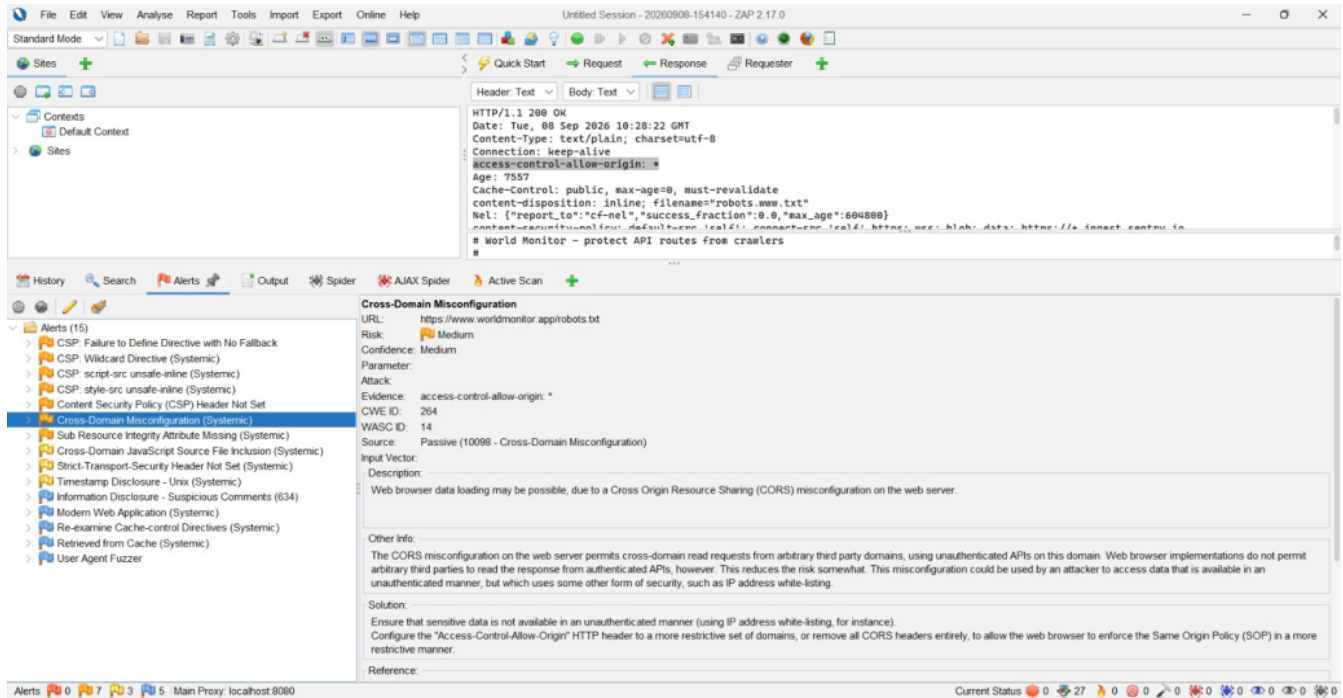


Figure 1: Matched Proof-of-Concept Screenshot for Vulnerability #5 (5/poc-1.png)

Authorization & Safety Compliance: This security assessment was conducted strictly under authorized parameters in accordance with ethical hacking standards. All validations and safe proof-of-concept tests were non-destructive. No persistent modifications or sensitive data exfiltration occurred.